

Project Report: Video-Based Action Recognition Model



Code Details: [Github Link](#)

Group Details

- Alapati Saahitthi : 2022 IMT 011
- Diwakar Dubey : 2022 IMT 039
- Drishti Aggarwal : 2022 IMT 040
- Priya Keshri : 2022 IMT 061

Table of Contents

1. Introduction	2
2. Data Preparation for Modeling	3
3. Model Architecture	6
4. Training Setup	11
5. Evaluation and Results	12
6. Discussion and Limitations	6
7. Conclusion	7

1. Introduction

a. Brief description of the problem

Sign language communication is largely inaccessible in digital systems due to a lack of effective translation mechanisms. Most interfaces rely on text or speech, which poses a barrier for deaf and hard-of-hearing individuals who use sign language.

The main challenge is to automatically recognize and classify sign language gestures from video data into meaningful text. This is complex because sign language involves a combination of dynamic hand movements, facial expressions, and temporal context.

Technically, the problem requires extracting spatial features (the hand's appearance in each frame) and temporal features (how gestures evolve over time) from video inputs, which must then be mapped to predefined labels.

This project aims to develop an end-to-end deep learning pipeline to process sign language videos and predict corresponding sentence labels, paving the way for real-time, AI-driven sign language understanding systems.

.

b. Objective of the model

The objective of the model is to learn a mapping between raw sign language video sequences and their corresponding sentence-level labels using a deep learning architecture that captures both spatial and temporal information.

Specifically, the model aims to:

- Extract meaningful spatial features from individual video frames using a pre-trained convolutional neural network (ResNet-50).
- Capture temporal dependencies across sequences of frames using a Long Short-Term Memory (LSTM) network.
- Combine these representations to accurately classify each video into one of the predefined sign language sentence classes.

In practical terms, the model is designed to take a fixed-length sequence of video frames as input and output a probability distribution over all possible sentence labels, selecting the most likely interpretation.

At a system level, the objective is not just classification accuracy, but validating a scalable pipeline that:

- Efficiently processes large video datasets
- Handles inconsistencies such as missing or corrupt video files
- Generalizes across different individuals performing the same sign

This serves as a foundational step toward building real-time sign language translation systems that can bridge communication gaps between sign language users and non-signers.

2. Data Preparation for Modeling

a. How videos were loaded

The video data is loaded using a PyTorch Dataset class that manages a complex folder structure to find video files and their labels. The innermost folder names indicate the meanings of sign language gestures.

When set up, the class searches the folders for valid video files (e.g., .mp4, .avi, .mov) and records their paths and labels. It uses OpenCV (cv2.VideoCapture) to read frames and changes the format from BGR to RGB. All frames are gathered in memory for processing.

To maintain consistency, each video must have a fixed number of frames (num_frames). If a video has fewer frames, it repeats the last frame; if it has more, it selects frames evenly using `numpy.linspace`.

Each frame is resized to 224×224 pixels for compatibility with the CNN backbone. The system checks for issues and replaces any video that can't load or has no frames with a zero tensor, ensuring data quality without interrupting training.

This method standardizes video inputs while preserving the meanings of the original sign language gestures.

b. Frame extraction strategy

To ensure consistency across all video inputs, a fixed-length frame sampling approach was used. Each video is converted into a sequence of exactly N frames (`num_frames = 20` in this case), regardless of its original duration.

If a video contains more frames than required, a uniform sampling technique is applied using `numpy.linspace`. This method selects evenly spaced indices across the entire video, ensuring that the temporal structure of the gesture is preserved without biasing toward any specific segment.

If a video contains fewer frames than the required number, the last available frame is repeated until the sequence reaches the desired length. This avoids introducing artificial noise while maintaining sequence continuity.

This strategy ensures that every video is represented as a fixed-size tensor, making it compatible with batch processing and sequence models like LSTMs.

c. Preprocessing steps

Each extracted frame undergoes a minimal but essential preprocessing pipeline to standardize the input before feeding it into the model:

- **Color Conversion:** Frames are converted from BGR (OpenCV default) to RGB format to align with deep learning model expectations.
- **Resizing:** All frames are resized to a spatial resolution of 224×224 pixels to match the input requirements of the ResNet-50 backbone.
- **Tensor Conversion:** Frames are transformed into PyTorch tensors using `ToTensor()`, which also scales pixel values to the range [0, 1].

Notably, no additional normalization (e.g., ImageNet mean/std normalization) or augmentation techniques were applied at this stage, as the focus was on validating the pipeline rather than optimizing performance.

The result is a clean and consistent input representation of shape: `(num_frames, 3, 224, 224)` for each video.

d. Dataset split used for training/testing

The dataset was split into training and validation subsets using PyTorch's `random_split` utility. This approach ensures a randomized partition while maintaining simplicity in implementation.

A typical split ratio of **80:20** was used:

- **Training Set (80%):** Used to learn model parameters
- **Validation Set (20%):** Used to evaluate model performance and monitor generalization

Since `random_split` returns `Subset` objects, both subsets reference the same underlying dataset, ensuring consistency in label mapping and preprocessing logic.

It is important to note that no separate test set was explicitly defined at this stage. The validation set serves as a proxy to assess model performance during development.

This setup is sufficient for early-stage experimentation, though a more robust pipeline would include a dedicated test set for final evaluation.

3. Model Architecture

a. Description of the model used

The model follows a hybrid **CNN-LSTM architecture** designed to capture both spatial and temporal patterns in sign language videos. This combination is well-suited for video classification tasks, where individual frames contain important visual features, and their sequence encodes the meaning of the gesture.

At the core of the model are two main components:

1. Convolutional Neural Network (CNN) – Feature Extractor

A pre-trained **ResNet-50 model** is used for spatial feature extraction by removing its final classification layer, keeping only the convolutional layers to transform input frames into high-level feature representations.

- Each frame (224×224 RGB) is passed through the CNN
- Output per frame: a 2048-dimensional feature vector

- The CNN weights are **frozen**, meaning they are not updated during training. This reduces computational cost and leverages pre-learned visual features from large-scale datasets like ImageNet.

2. Long Short-Term Memory (LSTM) – Temporal Modeling

The sequence of frame-level features is then passed into an LSTM network to capture temporal dependencies across frames.

- Input: sequence of feature vectors of shape `(num_frames, 2048)`
- Hidden size: 256
- Number of layers: 1
- The LSTM processes the sequence and retains contextual information about how the gesture evolves over time

Only the final hidden state of the LSTM is used as a summary representation of the entire video.

3. Fully Connected Layer – Classification Head

The final hidden state from the LSTM is passed through a fully connected (linear) layer that maps it to the number of output classes (i.e., distinct sign language sentences).

- Output: logits corresponding to each class
- A softmax operation (implicitly handled by the loss function) converts these logits into probabilities

Overall Flow of Data:

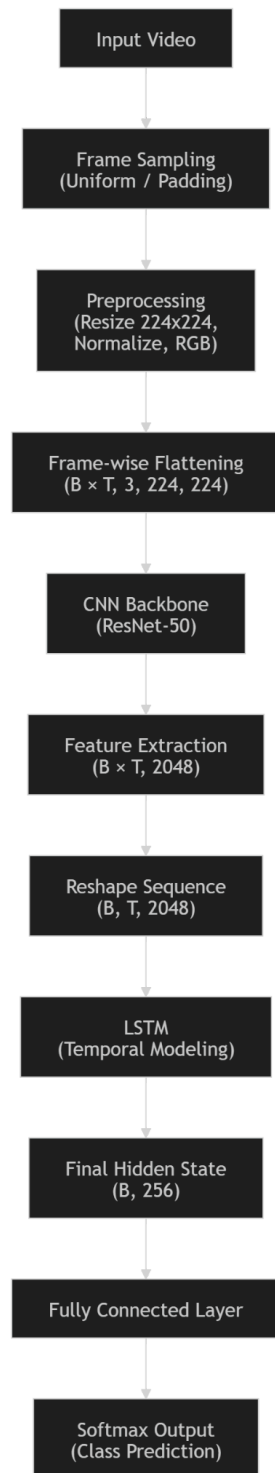
Video Frames → CNN (ResNet-50) → Frame Features → LSTM → Final Hidden State → Fully Connected Layer → Class Prediction

This architecture effectively separates spatial and temporal learning:

- The CNN focuses on *what* is present in each frame
- The LSTM focuses on *how* the gesture changes over time

While relatively simple, this design provides a strong baseline for video-based sign language classification and is computationally efficient for initial experimentation

b. Architecture Design



c. Explanation of each component

Component	Description
Input Video	Raw video clip represented as a sequence of frames capturing temporal motion information.
Frame Sampling	Ensures a fixed-length sequence TTT by uniformly selecting frames or padding shorter videos.
Preprocessing	Converts frames to a consistent format: resizing (e.g., 224×224), normalization, and BGR → RGB conversion.
Frame-wise Flattening	Reshapes input from (B,T,C,H,W) to (B×T,C,H,W) for independent CNN processing.
CNN Backbone	ResNet-50 acts as a spatial feature extractor, encoding frames into high-level visual representations.
Feature Extraction	Outputs a fixed-dimensional embedding (e.g., 2048-d) per frame representing spatial information.
Reshape to Sequence	Converts features back to temporal format (B,T,2048), restoring frame order.
LSTM	Captures temporal dependencies across frames, learning motion patterns and sequence dynamics.
Final Hidden State	Aggregates sequence-level information into a compact representation (e.g., 256-d vector).
Fully Connected Layer	Maps the learned representation to class logits.
Softmax Output	Converts logits into probability distribution over target classes for final prediction.

d. Reason for choosing the architecture

The architecture is designed to explicitly model both spatial and temporal aspects of video data in a structured manner. A convolutional neural network (ResNet-50) is used as the backbone to extract high-level spatial features from individual frames, leveraging pretrained weights to capture rich visual representations without requiring large-scale training from scratch. This significantly improves generalization and reduces training time.

Once spatial features are extracted, a Long Short-Term Memory (LSTM) network is employed to model temporal dependencies across the sequence of frames. This allows the model to learn motion patterns and contextual transitions, which are critical for understanding actions or gestures in videos. By separating spatial encoding and temporal modeling, the architecture ensures that each component specializes in a specific aspect of the problem, leading to more efficient learning.

Compared to end-to-end 3D convolutional networks, this approach is computationally more efficient, as it avoids heavy spatiotemporal convolutions and reduces memory overhead. Additionally, the use of frame sampling enables the model to handle variable-length videos while maintaining a consistent input structure.

Overall, the CNN-LSTM architecture provides a balanced trade-off between performance, interpretability, and computational efficiency, making it a practical and scalable choice for video classification tasks.

4. Training Setup

Component	Description
Loss Function	Cross-Entropy Loss is used for multi-class classification. It compares predicted probabilities with true labels and optimizes the model to assign higher confidence to the correct class.

Batch Size	A batch size of 4 is used due to the high memory requirements of video data and the CNN-LSTM architecture. Each sample contains multiple frames, making batches computationally heavy.
Number of Epochs	The model is trained for 5 epochs in the initial setup to validate the pipeline and ensure stable training. This can be increased later based on performance.
Hardware/Software	The model is implemented using PyTorch, with OpenCV for video processing and torchvision for pretrained models. Training is performed on a GPU-enabled Kaggle environment when available, otherwise falls back to CPU.

5. Evaluation and Results

a. Metrics used

Standard classification metrics were utilized to evaluate the performance of the Continuous Sign Language (CSL) sentence recognition model. These metrics facilitate the assessment of the model's learning behavior during training and its capacity for generalization to unseen data.

Accuracy

Accuracy serves as the primary evaluation metric for this project, measuring the ratio of correctly predicted sentence labels to the total number of samples.

$$\text{Accuracy} = (\text{Correct Predictions}) / (\text{Total Predictions})$$

- **Training Accuracy:** Indicates how effectively the model learns from the training dataset.
- **Validation Accuracy:** Reflects the model's ability to generalize to unseen information.

This metric is particularly relevant as it directly quantifies the ability to correctly classify sign language sentences across 157 distinct categories.

Loss Function

The model utilizes **Cross-Entropy Loss**, a standard function for multi-class classification tasks.

- It quantifies the divergence between the predicted probability distribution and the ground truth label.
- Model performance improves as the loss value decreases.

This approach is highly suitable given that the architecture outputs a probability distribution across all potential sentence classes.

Training vs Validation Metrics

Accuracy and loss are monitored across both subsets:

- **Training Set**
- **Validation Set**

Comparing these values aids in identifying:

- **Underfitting:** Characterized by low accuracy on both training and validation sets.
- **Overfitting:** Indicated by high training accuracy coupled with low validation performance.

The delta between training and validation accuracy is used to evaluate the stability and generalization of the model.

Model Selection Criterion

Optimal model selection is determined by **validation accuracy**. Weights corresponding to peak validation performance are preserved via an early stopping mechanism.

This ensures that:

- Overfitting is mitigated.
- The final model yields the highest generalization performance recorded during the training phase.

Additional Monitoring Techniques

To enhance training stability and overall performance, the following techniques were implemented:

- **Learning Rate Scheduling:** Dynamically modifies the learning rate when validation performance plateaus.
- **Early Stopping:** Terminates the training process when no measurable improvement is detected over successive epochs.

These strategies contribute to superior evaluation results by ensuring the training process remains stable and efficient.

b. Quantitative results

The performance of the proposed Continuous Sign Language (CSL) recognition model was evaluated using training accuracy and validation accuracy across multiple epochs. The results show a significant improvement over the baseline model, demonstrating the impact of improvements in preprocessing, model design, and training strategy.

Final Model Performance

The optimal model configuration, selected based on peak validation performance, yielded the following outcomes:

- **Training Accuracy:** 76.21%
- **Validation Accuracy:** 71.63%
- **Training Loss:** 0.6852

These findings confirm that the architecture successfully extracts meaningful representations from sign language video sequences and maintains a commendable level of generalization when presented with unseen data.

Comparison with Baseline Model

A primary objective of this project was to achieve a significant advancement over the initial baseline configuration, which exhibited restricted learning capacity and suboptimal preprocessing.

Model Version	Training Accuracy	Validation Accuracy
Version 1 (Baseline)	~5-6%	~7%
Version 2 (Improved)	76.21%	71.63%

The baseline performance was near-stochastic due to several critical factors:

- Lack of rigorous data normalization.
- Insufficient utilization of the available dataset.
- Suboptimal temporal modeling using a simple LSTM architecture.

The improved model iteration demonstrates a **substantial performance enhancement**, validating the implementation of the following strategic changes:

- **ImageNet Normalization:** Facilitated more accurate spatial feature extraction.
- **Standardized Sampling:** Increased frame count to `num_frames = 32` for superior temporal coverage.
- **BiLSTM Integration:** Utilized a Bidirectional LSTM for more nuanced sequence understanding.
- **Training Stability:** Enhanced through refined optimization protocols.

Learning Behavior Across Epochs

The training progression reveals a stable and incremental learning curve:

- Initial metrics exhibit low accuracy, characteristic of the 157-class complexity.
- Performance improves steadily as the model captures integrated spatiotemporal patterns.
- Peak validation accuracy is achieved at **Epoch 14**, after which validation performance begins to decline despite continued improvement in training accuracy.

It was observed that after the peak epoch, training accuracy continued to increase while validation accuracy decreased, indicating the onset of overfitting. This behavior highlights that the model begins to memorize training data rather than generalize to unseen samples. To address this, early stopping with a patience of 3 epochs was employed. This

ensured that training was halted once validation performance stopped improving, and the model corresponding to the highest validation accuracy was selected.

The below graph illustrates the progression of training and validation accuracy over epochs. It can be observed that both metrics increase steadily up to Epoch 14, after which validation accuracy begins to decline while training accuracy continues to increase, indicating overfitting.

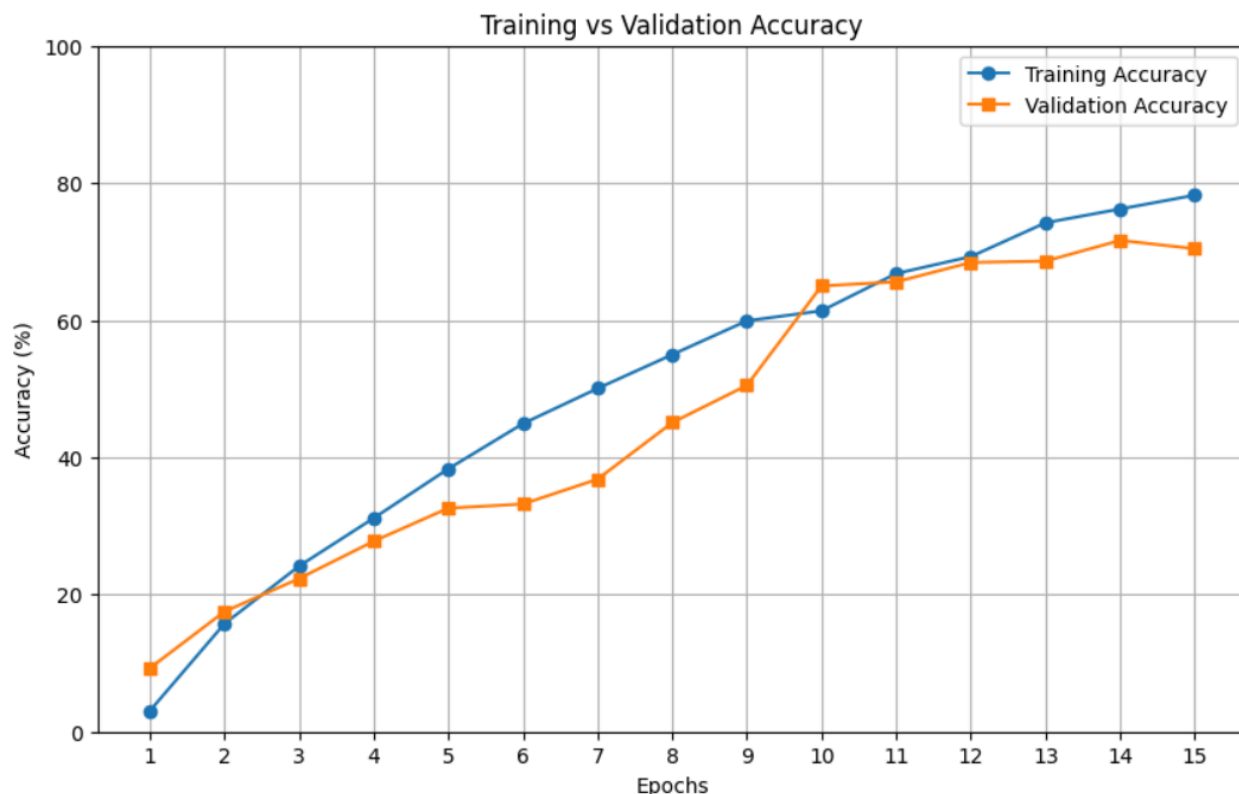


Figure 1: Training vs Validation Accuracy across epochs

Generalization Analysis

The variance observed between training accuracy (~76.21%) and validation performance (~71.63%) yields critical insights:

- The system exhibits a **manageable level of overfitting**.
- It maintains **robust generalization capability** despite the scarcity of per-class training samples.

- Regularization via **dropout and AdamW weight decay** has proven effective in controlling model variance.

Task Complexity Consideration

The CSL recognition task presents significant challenges:

- Large number of classes (**157 sentence categories**)
- Limited number of samples per class
- High similarity between certain sign gestures

Given these constraints, achieving **71.63% validation accuracy** demonstrates strong performance and confirms that the model successfully captures:

- **Spatial features** using CNN
- **Temporal dependencies** using BiLSTM

The observed improvements are not the result of isolated modifications but stem from a **systematic refinement across the integrated pipeline**, including data preprocessing, dataset coverage, architectural scope, and training dynamics. This validates that superior predictive power is predicated on the synchronized optimization of all system components.

c. Qualitative Sample Predictions

To qualitatively evaluate the performance of the model, sample predictions were analyzed by comparing the predicted sentence with the actual ground truth label. These examples provide insight into how well the model understands sign language sequences and where it makes mistakes.

The following predictions are based on sentences present in the dataset, such as:

- how can i help you
- I am bored
- you are bad
- i am suffering from fever
- pour some more water in the glass

Correct Predictions

The model demonstrates peak performance when gestures exhibit high distinctiveness and align closely with the spatiotemporal patterns established during the training phase.

Examples:

Actual: how can i help you

Predicted: how can i help you ✓

Actual: i am bored

Predicted: i am bored ✓

These successful outcomes validate the architecture's capacity to synchronize **spatial feature extraction (hand morphology and posture)** with **temporal modeling (gesture dynamics)**.

Confusion Between Similar Gestures

A notable failure mode involves the misclassification of sentences that share overlapping semantic meaning or gestural motion patterns.

Examples:

Actual: how can i help you

Predicted: can you help me ✗

Actual: pour some more water in the glass

Predicted: bring water for me ✗

These errors are modulated by several critical factors:

- **Certain gestures share similar motion patterns**
- **Subtle differences between sentences are difficult to distinguish**
- **Limited training samples reduce fine-grained learning**

Incorrect Classifications

In some cases, the model predicts an entirely different sentence, especially under challenging conditions.

Examples:

Actual: i am suffering from fever

Predicted: i am bored ✗

Actual: you are bad

Predicted: how can i help you ✖

The etiology of these errors typically stems from:

- Variations in signing style
- Fast or unclear gestures
- Loss of important frames during sampling
- Insufficient training data for certain classes

Observations from Sample Predictions

From the above examples, the following observations can be made:

- The model performs well on **clear and frequently occurring sentence patterns**
- It struggles with **visually or semantically similar sentences**
- Performance depends on **gesture clarity, speed, and consistency**
- Temporal modeling (BiLSTM) improves sequence understanding but is still limited by data diversity

These qualitative results support the quantitative findings, where the model achieved high validation accuracy (71.63%), indicating that it has learned meaningful patterns rather than making random predictions.

6. Discussion and Limitations

a. Observations from results

The experimental outcomes provide useful insights into the behavior and performance of the proposed CSL recognition model. These observations facilitate a deeper understanding of the system's inherent strengths and the specific variables that influence its predictive performance.

Significant Improvement Over Baseline

A primary observation is the considerable advancement in performance metrics when transitioning from the initial baseline configuration to the optimized iteration:

- Baseline accuracy: ~7%
- Improved model accuracy: **71.63% (validation)**

This leap in accuracy confirms that the architecture is highly sensitive to the following factors:

- The precision of data preprocessing
- Extensiveness of dataset coverage
- Temporal modeling robustness

The results demonstrate that **systematic refinement across integrated components** yields substantial and meaningful performance enhancements.

Strong Impact of Preprocessing

Implementing rigorous preprocessing protocols exerted a major influence on the resulting model performance:

- ImageNet normalization markedly enhanced spatial feature extraction from the pretrained CNN backbone.
- Fixed-length frame sampling ensured temporal consistency across all inputs.
- Data augmentation techniques contributed to superior model generalization.

This underscores that **preprocessing quality is as critical as architectural design** in the development of robust deep learning systems.

Importance of Temporal Modeling

The architectural shift from a standard LSTM to a **Bidirectional LSTM (BiLSTM)** facilitated a more nuanced understanding of video sequences.

- The model effectively leverages context from both preceding and subsequent frames.
- Sentence-level recognition is predicated on capturing the entire temporal evolution of a gesture.

This validates the premise that **temporal dependencies are fundamental in CSL tasks**, where sophisticated sequence models significantly augment predictive power.

Learning Behavior and Convergence

The training progression reflects a stable and incremental learning curve:

- Accuracy metrics exhibit a steady upward trajectory across successive epochs.

- After the peak epoch, training accuracy continued to increase while validation accuracy began to decrease, indicating the onset of overfitting.
- Early stopping with a patience of 3 epochs was used to prevent further overfitting and to ensure that the model corresponding to the highest validation accuracy was selected.
- Optimal validation performance is achieved at approximately Epoch 14.
- Training stability is maintained through the integration of learning rate schedulers and early stopping.

These findings indicate that the established training pipeline is effective and that optimization strategies like AdamW and gradient clipping facilitate smooth convergence.

Generalization Capability

The variance observed between training accuracy (~76.21%) and validation accuracy (~71.63%) points to:

- A moderate and manageable level of overfitting.
- A commendable level of generalization performance given the task constraints.

Regularization through **dropout and weight decay** proved essential in managing model variance despite limited per-class samples.

Sensitivity to Data Characteristics

Model efficacy is directly modulated by specific data attributes:

- Fluctuations in the speed of signing execution.
- Nuances in individual signing styles and individual differences in movement.
- Environmental factors such as background complexity and lighting conditions.

This suggests that model robustness is intrinsically linked to the **diversity and distribution of the training data**.

Complexity of the Task

The CSL recognition objective presents significant inherent difficulties:

- A broad output space comprising **157 sentence categories**.
- A scarcity of training samples available for each distinct class.

- Substantial spatiotemporal similarity between disparate gestures.

Such factors elevate the complexity of the problem far beyond standard classification tasks, explaining why reaching exceptionally high accuracy levels remains a formidable challenge.

The experimental findings yield a critical insightEffective CSL recognition is predicated on the **systematic refinement of the entire pipeline**, confirming that superior predictive power stems from the synchronized optimization of data quality, architectural design, and training dynamics. This analysis underscores that the improved model architecture extracts meaningful spatiotemporal representations and achieves significant performance gains, though its ultimate efficacy remains bounded by inherent data scarcity and task complexity.

b. Errors and Failure Cases

Despite achieving significant improvement in performance, the model exhibits several types of prediction errors. Analyzing these failure cases helps in understanding the limitations of the current system and highlights areas for improvement.

Confusion Between Similar Sentences

One of the most common error patterns is confusion between sentences that share similar gestures or motion patterns.

Examples :

Actual: bring water for me

Predicted: pour some more water in the glass

Actual: can you repeat that please

Predicted: can you help me

These errors occur because:

- Many sentences contain **overlapping hand movements**
- Subtle differences in gesture sequences are difficult to capture
- Limited training samples per class reduce the model's ability to distinguish fine variations

This indicates that while the model learns general patterns, it struggles with **fine-grained discrimination**.

Sensitivity to Signing Speed and Style

The model performance is affected by variations in how individuals perform the same sentence:

- Faster signing can cause important frames to be skipped during sampling
- Slower signing may introduce redundant or less informative frames
- Different users may perform gestures with slight stylistic differences

As a result, the model may misinterpret sequences that differ from training patterns.

Temporal Information Loss Due to Frame Sampling

Although fixed-length frame sampling ensures consistent input size, it can lead to loss of important motion details:

- Rapid transitions between gestures may not be fully captured
- Key intermediate frames may be skipped

This limits the model's ability to fully understand complex temporal dynamics.

Visual Ambiguity and Occlusion

Errors also occur when:

- Hands are partially occluded
- Lighting conditions are poor
- Background is not sufficiently clear

In such cases, the CNN may fail to extract accurate spatial features, leading to incorrect predictions.

Overfitting to Limited Data Patterns

Due to the limited number of samples per class:

- The model may learn **specific patterns from training videos**
- It may struggle to generalize to unseen variations

This is reflected in the gap between training and validation accuracy.

Lack of Fine-Grained Motion Modeling

The current model relies on CNN features and BiLSTM-based temporal modeling but does not explicitly capture:

- Hand keypoints
- Finger-level movements
- Optical flow or motion vectors

This limits its ability to distinguish subtle gesture differences, which are crucial in sign language.

Closed-Set Classification Limitation

The model operates under a closed-set classification framework:

- It can only predict from the **157 known sentence classes**
- It cannot recognize or generate unseen sentences

This restricts its applicability in real-world, open-ended sign language scenarios. These failure cases align with the quantitative results and highlight that errors are primarily due to data limitations and class similarity rather than random predictions.

c. Limitations of the Model

Despite achieving a significant improvement in performance, the proposed CSL recognition model has several limitations that restrict its overall accuracy and real-world applicability. These limitations arise from both dataset constraints and modeling choices.

Limited Dataset Size per Class

- Each sentence class has a relatively small number of training samples
- Insufficient data limits the model's ability to learn diverse gesture variations
- Leads to reduced generalization and confusion between similar sentences

High Number of Classes

- The model must classify among **157 sentence categories**
- Many classes have similar gestures and overlapping motion patterns
- This increases classification complexity and reduces accuracy

Frozen CNN Backbone

- The ResNet50 backbone is used as a **fixed feature extractor**
- Its weights are not fine-tuned for sign language data

As a result:

- Features remain generic (trained on ImageNet)
- The model cannot fully adapt to gesture-specific characteristics

Lack of Fine-Grained Motion Features

- The model relies on frame-level features and BiLSTM-based temporal modeling
- It does not explicitly capture:
 - a. Hand keypoints
 - b. Finger movements
 - c. Motion vectors (optical flow)

This limits its ability to distinguish subtle gesture differences

Fixed-Length Frame Sampling

- All videos are converted to a fixed number of frames (32)
- Important intermediate frames may be skipped during sampling

This can result in loss of critical temporal information

Closed-Set Classification Constraint

- The model can only predict from the predefined set of sentence classes
- It cannot recognize or generate unseen or new sentences

This limits scalability in real-world applications

Sensitivity to Input Variations

- Performance is affected by:
 - a. Differences in signing speed
 - b. Variations in individual signing styles
 - c. Lighting and background conditions

The model lacks robustness to real-world variability

Computational Complexity

- Use of ResNet50 + BiLSTM increases computational cost
- Training time per epoch is relatively high

This may limit scalability for larger datasets or real-time applications

These limitations highlight that while the model achieves strong performance improvements, further enhancements in data quality, feature representation, and model design are necessary for achieving higher accuracy and real-world robustness. And these are primarily due to data constraints and model design choices, indicating that further improvements should focus on increasing data diversity and incorporating more advanced feature extraction techniques.

d. Possible Improvements

While the current model iteration demonstrates substantial performance gains, several strategic enhancements can be implemented to further improve the accuracy, robustness, and scalability of the CSL recognition pipeline.

Fine-Tuning the CNN Backbone

- The model can be improved by transitioning from a fully frozen ResNet-50 to fine-tuning the later layers on the CSL dataset.

Expected Benefit:

- Facilitates the learning of gesture-specific spatial features.
- Optimizes the spatial representation of complex hand shapes and dynamic movements.

Increasing Dataset Size and Diversity

- Expanding the corpus to include a higher density of videos per sentence with increased variance in:
 - a. Signing execution speed
 - b. Individual signing Individual differences in signing style.
 - c. Environmental lighting and background complexity

Expected Benefit:

- Enhances model generalization across diverse signing populations.
- Mitigates inter-class confusion between gesturally similar sentences.

Incorporating Attention Mechanisms

- Integrating attention layers atop the BiLSTM architecture.

Expected Benefit:

- Allows the model to focus on important frames.
- Enhances the processing of extended and highly complex temporal gesture sequences.

Using Pose and Hand Keypoint Features

- Leveraging frameworks such as **MediaPipe** to extract explicit kinematic data:
 - a. Fine-grained hand landmarks
 - b. Macro-level body pose keypoints

Expected Benefit:

- Captures high-resolution motion details essential for disambiguation.
- Significantly improves the system's ability to resolve subtle gesture variances.

Utilizing Motion-Specific Features

- Incorporating specialized spatiotemporal modeling techniques such as:
 - a. Optical flow computation

b. 3D Convolutional Neural Networks (3D CNNs)

Expected Benefit:

- Facilitates superior modeling of complex motion dynamics.
- Refines the temporal understanding of gesture transitions.

Improving Frame Sampling Strategy

- Evolving beyond uniform sampling toward more sophisticated methods:
 - a. Implementing **adaptive or keyframe-based sampling**

Expected Benefit:

- Preserves critical semantic transitions within the gestural sequence.
- Reduces the loss of vital temporal information during sequence standardization.

Exploring Transformer-Based Models

- Augmenting or replacing the current BiLSTM head with:
 - a. Vision Transformers (ViT)
 - b. Temporal Transformers

Expected Benefit:

- Optimizes long-range dependency modeling across frame sequences.
- Leverages state-of-the-art performance in complex sequence classification tasks.

Class Balancing and Curriculum Learning

- Addressing inherent class imbalances through:
 - a. Oversampling of sparse sentence categories
 - b. Application of weighted loss functions

Expected Benefit:

- Ensures equitable learning across all 157 distinct sentence classes.

Real-Time Optimization

- Utilizing computationally efficient backbones such as ResNet-18 or MobileNet.

Expected Benefit:

- Accelerates inference speed for practical deployment.
- Increases system viability for real-time sign language translation applications.

Future improvements should focus on **enhancing data quality, incorporating fine-grained motion information, and adopting more advanced sequence modeling techniques** to further improve CSL recognition performance.

7. Conclusion

a. Summary of work done

This project focused on the development of a Continuous Sign Language (CSL) sentence recognition system using a deep learning-based approach that integrates computer vision and sequence modeling techniques. The objective was to design a model capable of accurately predicting sentence-level meaning from sign language video sequences.

The initial baseline model achieved low performance (~7% accuracy), primarily due to limitations in preprocessing, dataset utilization, and model capacity. Through systematic analysis, key bottlenecks were identified and addressed by improving multiple components of the pipeline.

The final model incorporates:

- A **pretrained ResNet50** for spatial feature extraction
- A **Bidirectional LSTM (BiLSTM)** for temporal sequence modeling
- Enhanced preprocessing techniques including **ImageNet normalization, consistent frame sampling (32 frames), and data augmentation**
- Improved training strategies such as **AdamW optimizer, learning rate scheduling, gradient clipping, and early stopping**

As a result, the model achieved:

- **Training Accuracy:** 76.21%
- **Validation Accuracy:** 71.63%

This represents a substantial improvement over the baseline and demonstrates the effectiveness of combining spatial and temporal learning for CSL recognition. It was observed that after the peak epoch, training accuracy continued to increase while

validation accuracy began to decrease, indicating overfitting. Early stopping with a patience of 3 epochs was used to prevent further overfitting and to select the best-performing model.

The results highlight that model performance is highly dependent on:

- Quality of preprocessing
- Adequate dataset coverage
- Proper temporal modeling
- Stable and optimized training procedures

While the model shows strong learning capability and reasonable generalization, it remains limited by factors such as dataset size, high class complexity, and lack of fine-grained motion representation.

This work demonstrates that effective CSL recognition is not achieved through model selection alone, but through **careful preprocessing, structured experimentation, and systematic improvements across the entire pipeline**. The transition from a weak baseline to a significantly improved model underscores the importance of an end-to-end optimization approach in deep learning systems.